

Conditional Execution

Conditional execution, or control flow, is pivotal to programming, including JavaScript. It enables code to respond dynamically to varying situations rather than behaving uniformly. This flexibility is essential for building interactive and responsive programs. Think of your program as a tree: the trunk is the starting point, and each conditional instruction represents a branching path based on decisions, calculations, or input.

JavaScript offers various ways to manage code branching through conditions. From here on, you'll encounter more hands-on tasks and code examples as we explore these concepts.

The if Statement

The if statement is JavaScript's simplest control flow mechanism. It evaluates a condition and, if true, executes the code block following it. The syntax is:

javascript

Copy code

```
if (condition) {  
    // Code block to execute if condition is true  
}
```

Here, the if keyword is followed by a condition in parentheses. If the condition evaluates to true, the code inside the curly brackets runs. Otherwise, it's skipped. For example:

javascript

Copy code

```
let isUserReady = confirm("Are you ready?");  
console.log(isUserReady);  
if (isUserReady) {  
    alert("User ready!");  
}
```

In this example, the alert is shown only if the user clicks "OK" in the confirmation dialog.

Best Practices for if Statements

While a single statement inside the if block can omit curly braces, always using braces is recommended. This avoids errors when adding more statements later. Consider this example:

javascript

Copy code

```
let isUserReady = confirm("Are you ready?");
```

```
if (isUserReady)
  console.log("User ready!");
  alert("User ready!");
```

Here, only the first line is part of the if block. The second line runs regardless. Fix this by enclosing both statements in braces:

javascript

Copy code

```
if (isUserReady) {
  console.log("User ready!");
  alert("User ready!");
}
```

Scope and Variables in if Statements

Variables declared with `let` or `const` inside an if block are scoped to that block. Attempting to access them outside results in an error:

javascript

Copy code

```
let unitPrice = 10;

let pieces = prompt("How many pieces do you order?", 0);
```

```
if (pieces > 0) {
  let total = unitPrice * pieces;
  console.log(total);
}
```

```
console.log(total); // -> Uncaught ReferenceError: total is not defined
```

Using Logical and Comparison Operators

Conditions often use logical and comparison operators for more complex checks. For example:

javascript

Copy code

```
let userAge = 23;
```

```
let points = 703;
```

```
let cartValue = 299;
```

```
let shippingCost = 9.99;
```

```
if (userAge > 21 && (cartValue >= 300 || points >= 500)) {  
    shippingCost = 0;  
}
```

```
console.log(shippingCost);
```

Here, shipping is free if the user is over 21 **and** the cart value is at least 300 **or** they have 500 or more points.

The if ... else Statement

For scenarios requiring alternate actions when a condition is false, use the else keyword:

javascript

Copy code

```
let isUserReady = confirm("Are you ready?");
```

```
if (isUserReady) {  
    console.log("User ready!");  
} else {  
    console.log("User not ready!");  
}
```

```
}
```

This structure ensures exactly one block executes.

The if ... else if Statement

For multi-branch logic, use if ... else if statements. For instance:

javascript

Copy code

```
let number = prompt("Enter a number", 0);
```

```
if (number < 10) {  
    alert("<10");  
} else if (number < 30) {  
    alert("<30");  
} else if (number < 60) {  
    alert("<60");  
} else if (number < 90) {  
    alert("<90");  
} else if (number < 100) {  
    alert("<100");  
} else {  
    alert(">100");  
}
```

JavaScript evaluates conditions sequentially and stops after the first true condition.

The Conditional (Ternary) Operator

The conditional operator (?:) is a shorthand for if ... else. It's useful for simple conditional assignments:

javascript

Copy code

```
let price = 100;
```

```
let shippingCost = price > 50 ? 0 : 5;
```

```
console.log(`price = ${price}, shipping = ${shippingCost}`); // -> price = 100, shipping = 0
```

This concise approach enhances readability for straightforward conditions.